

Stereo Display

Project Documentation

Raspberry Pi stereo display with BluOS artwork mode, analog recognition, Spotify metadata correction, and Last.fm scrobbling.

Current display	Small composite CRT
Main app	stereo_display.py
Runtime path	/home/spieg16/repos/stereo-display
Analog recognition	ACRCloud via USB audio sample
Metadata + artwork	Spotify correction + BluOS/iTunes fallback
Scrobbling	Last.fm Now Playing refresh + track/end-of-side scrobbles
Mode control	SayoDevice 1x1P with LED indicator

Last updated: 2026-06-30

Overview

Stereo Display is a Raspberry Pi project that provides a full-screen now-playing display for a BluOS player while also supporting analog source recognition. It was built for a small composite CRT monitor and optimized around CRT constraints: overscan, soft text, non-square displayed pixels, and limited usable screen space.

The current application has two operating modes:

- ART Mode - displays BluOS album artwork from the BluOS local API.
- ANALOG Mode - monitors an analog input, identifies music with ACRCLOUD, improves metadata with Spotify when safe, retrieves artwork, updates Last.fm Now Playing, scrobbles tracks, and returns to ART Mode after silence.

The app is primarily run from `stereo_display.py`.

Current Hardware Setup

Required hardware:

- Raspberry Pi running the desktop environment. The current build has been used on a Raspberry Pi 3B+.
- BluOS-compatible player, tested with a Bluesound Node.
- USB audio capture interface, tested with TI PCM2902 / Behringer UCA202 / UCA222 style devices.
- Network connectivity.
- Small composite CRT display.

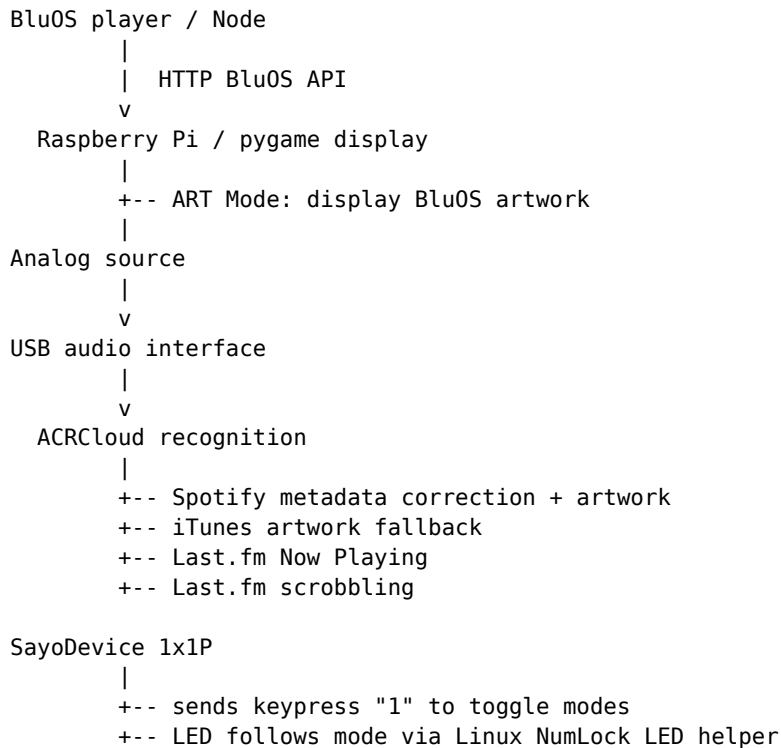
Optional but currently used in this build:

- SayoDevice 1x1P macro key for mode switching.
- Turntable, tuner, tape deck, or another analog source feeding the USB audio interface.

Future display note:

- HDMI LCD support is not part of the current setup. The code includes comments describing likely future HDMI migration changes, but the current layout is tuned for a composite CRT.

High-Level Architecture



Operating Modes

ART Mode

ART Mode is the default startup mode. It polls the BluOS player and displays the currently available artwork full-screen within a CRT-safe area.

Key behavior:

- Uses the BluOS /Status endpoint.
- Tracks etag values to avoid unnecessary redraws.
- Downloads only the artwork field needed for the display.
- Uses letterboxing so artwork is not cropped.

- Applies CRT pixel-aspect compensation.
- Turns the Sayo LED off.

ANALOG Mode

ANALOG Mode is entered by pressing the Sayo key, which sends the 1 keypress.

Current workflow:

1. User activates ANALOG Mode.
2. Sayo indicator LED turns on.
3. Manual entry resets the initial-recognition cooldown, Last.fm refresh timer, and provisional-track state.
4. The app waits for audio using short RMS-check samples.
5. Once audio is detected, the app waits briefly before fingerprinting.
6. A 20-second initial sample is sent to ACRCLOUD.
7. If recognition fails, a 30-second fallback sample is attempted.
8. If recognized, the result is passed through conservative Spotify metadata correction.
9. The active title is cleaned before display, internal comparison, and Last.fm submission.
10. The app retrieves artwork, sends Last.fm Now Playing, and displays the track.
11. During playback, the app records 12-second recheck samples every 30 seconds.
12. Track changes scrobble the previous track when it is eligible.
13. Repeated silence scabbles the final eligible track and returns to ART Mode.
14. The Sayo LED turns off.

Source Files

`stereo_display.py`

Main application and event loop.

Responsibilities:

- Pygame initialization and full-screen rendering.
- ART / ANALOG mode switching.
- BluOS polling and artwork rendering.
- Analog recognition workflow orchestration.
- Silence detection and retry timing.
- Last.fm Now Playing refreshes and scrobble calls.
- Spotify metadata correction orchestration.
- Sayo LED integration.
- CRT-safe layout.

Important constants:

```

NODE_IP = "192.168.4.40"
SAFE_MARGIN_X_RATIO = 0.08
SAFE_MARGIN_Y_RATIO = 0.10
PIXEL_ASPECT_X = 1.50
ANALOG_RECHECK_SECONDS = 30
LASTFM_NOW_PLAYING_REFRESH_SECONDS = 120
SILENCE_THRESHOLD = 900
SILENCE_CHECKS_BEFORE_EXIT = 3
INITIAL_RECOGNITION_RETRY_SECONDS = 45
MIN_ACR_SCORE_FOR_DIFFERENT_ARTIST_CHANGE = 95
MIN_UNCONFIRMED_TRACK_AGE_BEFORE_SCROBBLE_SECONDS = 45
MIN_UNCONFIRMED_TRACK_DURATION_FRACTION_BEFORE_SCROBBLE = 0.50
MIN_UNCONFIRMED_TRACK_AGE_FLOOR_BEFORE_SCROBBLE_SECONDS = 20

```

Current recording behavior in `stereo_display.py`:

```

def record_sample(seconds=12):
    ...

```

The default 12-second sample is used for ongoing rechecks. Initial recognition explicitly records 20 seconds, and fallback recognition explicitly records 30 seconds.

analog_recognition.py

Handles ACRCLOUD fingerprint recognition.

Responsibilities:

- Uses `ACRCLOUDRecognizer`.
- Reads `/tmp/current_sample.wav`.
- Parses the best ACRCLOUD music match.

- Rejects very low-confidence ACRCLOUD matches.
- Returns normalized fields used by the main app.

Returned fields include:

```
title
artist
album
score
duration_ms
spotify_album_id
spotify_track_id
isrc
```

Important behavior:

- ACRCLOUD's first/best music match is used.
- Results below `MIN_ACR_SCORE = 35` are rejected.
- The main app decides whether a recognized result is the same track, a track change, or a failed identification.
- The helper `record_sample(seconds=18)` in this file is for standalone testing. Production recording is controlled by `stereo_display.py`.

album_art.py

Handles Spotify metadata correction, artwork lookup, and artwork download.

Responsibilities:

- Gets a Spotify client-credentials token.
- Caches the Spotify token in memory.
- Uses Spotify album IDs when available.
- Corrects ACRCLOUD metadata with a conservative Spotify lookup.
- Protects live, unplugged, and concert recordings from studio normalization.
- Falls back to iTunes artwork search when Spotify artwork is unavailable.
- Downloads artwork as RGB PIL images.

lastfm.py

Handles signed Last.fm API writes.

Responsibilities:

- Computes Last.fm API signatures.
- Sends `track.updateNowPlaying`.
- Sends `track.scrobble`.
- Logs API error responses when available.

Project Directory

Intended runtime path on the Pi:

`/home/spiegl6/repos/stereo-display`

Current ZIP contents:

```
README.md
README.pdf
.gitignore
acr_config.py
album_art.py
analog_recognition.py
archive/artwork_only_display.py
archive/artwork_with_text_display.py
archive/lastfm_auth.py
assets/stereo-display.desktop
assets/logrotate.d/stereo-display
assets/sayo-led
stereo_display_dependencies.txt
stereo_display.py
lastfm.py
lastfm_config.py
requirements.txt
spotify_config.py
```

The config files in the shared archive are redacted. The virtual environment and runtime log are intentionally not part of the clean project snapshot.

Dependencies

OS Packages

Install with apt:

```
sudo apt install python3 python3-venv python3-pip alsa-utils sox libasound2-dev libhidapi-  
hidraw0 logrotate numlockx vim
```

Package notes:

- `alsa-utils` provides `arecord`.
- `logrotate` manages the runtime log.
- `numlockx` was useful during Sayo LED testing.
- `libhidapi-hidraw0` was used during HID investigation and may be useful for future Sayo work.
- `vim` is included because this project is maintained directly on the Pi.

Python Packages

From `requirements.txt`:

```
acrcloud==14.10.2020
certifi==2026.2.25
charset-normalizer==3.4.5
idna==3.11
numpy==2.4.6
pillow==12.1.1
pyacrcloud==1.0.11
pygame==2.6.1
requests==2.32.5
urllib3==2.6.3
```

Optional/development package:

`hidapi`

`hidapi` is not imported by the current production app. It was used only for Sayo HID experiments.

Configuration Files

The config files are intentionally separate from the main code so secrets can be redacted, regenerated, or replaced without editing the application logic.

`acr_config.py`

Required variables:

```
ACR_HOST = "..."  
ACR_ACCESS_KEY = "..."  
ACR_ACCESS_SECRET = "..."
```


spotify_config.py

Required variables:

```
SPOTIFY_CLIENT_ID = "..."  
SPOTIFY_CLIENT_SECRET = "..."
```

lastfm_config.py

Required variables:

```
LASTFM_API_KEY = "..."  
LASTFM_SHARED_SECRET = "..."  
LASTFM_SESSION_KEY = "..."
```

Audio Capture

The current code records from:

```
plughw:2,0
```

using:

```
arecord -D plughw:2,0 -f cd -d <seconds> /tmp/current_sample.wav
```

If the USB audio interface appears as a different ALSA device, update `record_sample()` in `stereo_display.py` and, if used directly, `record_sample()` in `analog_recognition.py`.

Recognition Behavior

The app uses different sample lengths for different purposes:

- 2 seconds - quick audio presence checks while waiting for audio.
- 12 seconds - ongoing analog rechecks for silence and track-change recognition.
- 20 seconds - initial ACRCLOUD fingerprint attempt after audio is first detected.
- 30 seconds - fallback ACRCLOUD fingerprint attempt when the initial attempt fails.

The 12-second recheck sample replaced earlier 4-second rechecks. Four seconds was fast and responsive but proved too vulnerable to sampled material and short-loop false positives.

If an initial recognition attempt fails, the app waits 45 seconds before another ACRCLOUD attempt. During that cooldown, the app still records short monitor samples and logs the remaining wait time.

Silence Detection

Silence detection is RMS-based.

Current threshold:

```
SILENCE_THRESHOLD = 900
```

Real-world observations during tuning:

- True silence: very low single digits.
- Runout / low noise: low hundreds.
- Music: often 2000+.

The app requires multiple consecutive silent checks before returning to ART Mode:

```
SILENCE_CHECKS_BEFORE_EXIT = 3
```

Spotify Metadata Correction

Spotify metadata correction is a best-effort cleanup layer that runs after ACRCLOUD has identified the audio.

Design principle:

ACRCLOUD identifies the recording.

Spotify improves the metadata only if it can do so confidently.

The correction layer exists because ACRCLOUD often identifies the right track but attaches it to a compilation, anthology, budget reissue, deluxe edition, or otherwise undesirable release.

Examples of problems it is meant to reduce:

- ACR album: Beautiful Brother - The Essential
- ACR album: Dancefloor Jazz
- ACR album: Broken Colour

- ACR album: unrelated compilation titles
- ACR title: Teen Age Riot (Album Version)
- ACR title: Those Shoes (LP Version)

Spotify Search and Scoring

album_art.py searches Spotify using the ACRCLOUD artist and title.

Candidate scoring:

```
50 points - exact track title match
35 points - exact primary artist match
15 points - Spotify album_type == album
20 points - meaningful album-word overlap with the ACRCLOUD album
-30 points - obvious compilation album penalty
```

Only candidates scoring at least 85 are eligible.

The album-word overlap bonus is intentionally not a hard requirement. ACRCLOUD can identify the correct recording with messy compilation/reissue metadata, so Spotify should still be able to improve those cases when it has a better album candidate.

Obvious compilation albums can still win when no better full-album candidate exists, but they are penalized so best of, greatest hits, essential, anthology, collection, compilation, and singles releases do not beat plausible original-album candidates too easily.

Primary Artist Safeguard

Spotify corrections are allowed only when the primary Spotify artist matches the ACRCLOUD artist after narrow normalization.

The normalization handles harmless catalog differences such as:

```
The Allman Brothers Band
-> Allman Brothers Band

_GEORGE_HARRISON
-> George Harrison
```

The gate still requires the primary Spotify artist to match. It does not accept fuzzy matches or cases where the ACRCLOUD artist appears only as a featured or secondary Spotify artist.

This prevents title-only false corrections such as:

```
Curtis Mayfield - Give Me Your Love
```

-> \$heem - GIVE ME YOUR LOVE

If Spotify cannot find a trustworthy match, the original ACRCLOUD result is returned unchanged. This protects obscure records and records that are not available on Spotify.

Protected Recordings

If the ACRCLOUD album or title says the recording is live, unplugged, or concert material, Spotify correction is skipped.

This prevents a live recording such as:

Badlands (Live at Madison Square Garden, New York, NY - June/July 2000)

from being normalized to a studio release.

When this protection triggers, embedded ACRCLOUD Spotify IDs are removed before artwork lookup. Those embedded IDs can point to the studio album even when the title or album clearly identifies a live recording.

Title Matching for Spotify Lookup

Spotify sometimes includes descriptive parentheticals that ACRCLOUD omits, such as:

Give Me Your Love
Give Me Your Love (Love Song)

For Spotify candidate matching only, the code normalizes titles enough to treat safe descriptive aliases as equivalent. It does not collapse meaningful variants such as Live, Reprise, Part, Alternate Take, Demo, Mix, Edit, Instrumental, Mono, or Stereo.

Metadata Cleanup for Display and Last.fm

After ACRCLOUD and any Spotify correction, the active title and artist are cleaned and stored back into the result used by the CRT display, internal track comparison, and Last.fm.

Artist cleanup treats underscores as spaces so catalog-style ACRCLOUD names display normally:

_GEORGE_HARRISON
-> George Harrison

The title cleanup removes generic suffixes such as:

Album Version
LP Version
Remaster
Remastered
Remastered Version
2013 Remaster
2001 Digital Remaster

It does not remove meaningful text such as:

Love Song
Live
Part 2
Alternate Take
Reprise

The album display cleanup removes generic release-edition parentheticals and suffixes such as:

2021 Remaster
Deluxe Edition
Expanded Edition
Expanded Reissue
Bonus Track Edition
Special Edition
Legacy Edition
- 40ème Anniversaire

It preserves meaningful album parentheticals such as:

Original Soundtrack
White Album

Spotify Album Selection

When multiple Spotify candidates are good matches, the code prefers:

1. Highest score.
2. Earliest release date.
3. Full albums over singles and EPs.
4. Shorter album title when everything else is effectively equal.

This is intended to favor original albums over compilations, singles, EPs, deluxe editions, and later reissues when possible, but it remains heuristic rather than Discogs-level release matching.

Artwork Behavior

Analog artwork lookup uses a layered approach:

1. Use a Spotify album ID when the result has one.
2. If Spotify artwork is unavailable, fall back to iTunes artwork search.
3. Prefer album title over track title for iTunes fallback search.

The album-first fallback matters for live and compilation-prone material. Track-title searches can land on the wrong release when the same song appears on a studio album, live album, compilation, or remaster.

Artwork is cached in memory during the current app run. Cache keys use Spotify album ID when available, otherwise artist / album / title.

Last.fm Behavior

The app uses Last.fm in three ways:

- Now Playing is sent when a track is first recognized or when a track changes.
- Now Playing is refreshed every 120 seconds while an identified analog track is active.
- Scrobbles are sent when a track change occurs or when silence returns the app to ART Mode.

The final track on a side is scrobbled before ART Mode resumes if it is eligible.

Last.fm does not provide a normal API method to explicitly clear Now Playing. When the app returns to ART Mode, it stops refreshing Now Playing and waits for Last.fm to expire the state on its side.

Provisional Track Scrobbling

The first recognized track in an analog session starts as provisional. It can display and update Last.fm Now Playing immediately, but it is not scrobbled until it is eligible.

A provisional track becomes eligible when either:

- a later recheck confirms the same track, or
- it has remained active long enough.

The unconfirmed age requirement is adaptive:

`required seconds = min(45, max(20, duration_seconds * 0.50))`

This protects against high-confidence false initial recognitions while still allowing short tracks to scrobble when ACRCLOUD provides a duration.

Scrobble Deduplication

The app remembers the most recently scrobbled normalized track key. This prevents immediate duplicate scrobbles of the same track.

The dedupe is not a full set of every track in a session. If the same track appears again later after another track intervenes, it can scrobble again.

Track Change Protection

Track comparison normalizes some ACR/metadata variants so these are not treated as different tracks:

- Album Version
- LP Version
- Remaster
- Remastered
- Remastered Version

For internal track identity only, a trailing live-location parenthetical is treated as metadata rather than a different song. This prevents ACRCLOUD from creating a false track change when it alternates between:

Badlands (Live at Madison Square Garden, New York, NY - June/July 2000)
Badlands

The live detail is still preserved for display and Last.fm when ACRCLOUD provides it.

A separate protection exists for different-artist changes.

Current threshold:

`MIN_ACR_SCORE_FOR_DIFFERENT_ARTIST_CHANGE = 95`

Reason:

- Same-artist track changes are normal on LP playback.

- Different-artist changes during one analog session are more suspicious.
- Sampled material can cause ACRCLOUD to identify the later song that sampled the current record.

A different-artist track change below 95 is rejected and logged.

SayoDevice Integration

The SayoDevice 1x1P acts as the physical mode switch. It sends a keypress that pygame receives as K_1.

Expected LED behavior:

- ART Mode - LED off.
- ANALOG Mode - LED on.

The Sayo LED is controlled by a helper script at runtime:

```
/usr/local/bin/sayo-led
```

The helper script writes to the current Linux NumLock LED brightness device:

```
/sys/class/leds/input*::numlock/brightness
```

The helper requires a sudoers entry allowing passwordless execution:

```
spieg16 ALL=(root) NOPASSWD: /usr/local/bin/sayo-led
```

The Linux LED device name can change across reboots or USB reconnects, e.g. input18::numlock may become input23::numlock. The helper script should search dynamically rather than hardcoding the input number.

Startup

The app has been launched through desktop autostart. A working autostart command uses unbuffered Python and appends output to the log:

```
Exec=/bin/bash -c '/home/spieg16/repos/stereo-display/venv/bin/python -u /home/spieg16/repos/stereo-display/stereo_display.py >> /home/spieg16/repos/stereo-display/stereo-display.log 2>&1'
```

The -u flag is important. It keeps Python output unbuffered so tail -f shows current log entries promptly.

The generated user-service restart command has not been reliable on the current Pi.
The normal working deploy cycle is to format edited files with `black` and `reboot`:

```
black ~/repos/stereo-display/stereo_display.py ~/repos/stereo-display/album_art.py
sudo reboot
```

Logging

Runtime log:

`~/repos/stereo-display/stereo-display.log`

Recommended logrotate configuration:

```
/home/spieg16/repos/stereo-display/stereo-display.log {
    daily
    rotate 3
    compress
    copytruncate
    missingok
    notifempty
}
```

Why `copytruncate` matters: the Python process keeps the log file open. `copytruncate` rotates the file without needing to restart the app.

Useful commands:

```
tail -f ~/repos/stereo-display/stereo-display.log
black ~/repos/stereo-display/stereo_display.py ~/repos/stereo-display/album_art.py
sudo reboot
```

Typical Log Messages

```
Application started in mode: art
Mode changed to: analog
Sample RMS: 2126
Audio detected. Waiting before identification...
ACR Match: Sonic Youth - Teen Age Riot (Album Version) (album=Daydream Nation (Deluxe Edition),
score=100)
Now Playing: Sonic Youth - Teen Age Riot (Daydream Nation)
Last.fm Now Playing: Sonic Youth - Teen Age Riot
Confirmed analog track: Sonic Youth - Teen Age Riot
Track Change: Eagles - Those Shoes (score=100)
Last.fm Scrobble: Eagles - Heartache Tonight
Last.fm Now Playing: Eagles - Those Shoes
Last.fm Now Playing Refresh: Eagles - Those Shoes
Silence count: 3
No audio detected. Switching back to ART mode.
```

Known Quirks

ACRCloud misses obscure or long-form material

Some records are not fingerprinted well. The app may log repeated ACR status: No result messages even when RMS shows clear audio. The 30-second fallback helps some records but cannot fix missing or poor database coverage.

ACRCloud can produce false matches

False matches are possible, especially on records that have been heavily sampled, sparse jazz passages, runout grooves, or very short recognition windows. Current mitigations include RMS gating, longer initial samples, 12-second rechecks, low-score filtering, different-artist confidence thresholds, title normalization, provisional scrobble protection, and session-level scrobble deduplication.

Spotify correction is conservative, not authoritative

Spotify metadata correction is deliberately best-effort. If Spotify cannot confidently identify the same artist/title, the original ACRCloud result is retained. This matters for rare, private, bootleg, regional, or non-streaming records.

Album metadata can still vary

Even after Spotify correction, album naming can vary across releases. The current logic strongly improves the most common problems but does not guarantee Discogs-level release accuracy.

Compilation and release selection remain heuristic

The code penalizes obvious compilations and rewards album-word overlap, but it can still choose an undesired release when Spotify search results are incomplete, ACRCloud album metadata is wrong, or multiple releases look equally plausible.

Button responsiveness depends on sample length

Any call to arecord blocks the pygame loop. Short 2-second samples are used while waiting for audio so mode switching stays responsive. Longer samples are used when

identification quality matters.

Last.fm Now Playing clears on Last.fm's side

When returning to ART Mode, the app stops refreshing Now Playing. The old Now Playing entry can remain visible until Last.fm expires it.

CRT-Specific Notes

The current UI is tuned for a small composite CRT:

- Safe-area margins protect against overscan.
- Text is intentionally large.
- Artwork is placed conservatively.
- Analog cover art is drawn wider than tall so it appears visually correct through the composite/CRT chain.
- Long analog titles are capped at two visible lines and truncated with an ellipsis.

Future HDMI Migration

If moving to a small HDMI LCD later, likely changes include:

- Reduce safe margins.
- Use square album art.
- Revisit font sizes.
- Move status text closer to the screen edge.
- Add more metadata or status indicators.
- Consider a separate HDMI-specific layout mode.

Rebuild Checklist

1. Install OS dependencies.
2. Create and activate venv.

3. Install `requirements.txt`.
4. Copy or recreate redacted config files with real credentials.
5. Confirm BluOS player IP in `stereo_display.py`.
6. Confirm ALSA input device with `arecord -l`.
7. Install or recreate the Sayo LED helper at `/usr/local/bin/sayo-led`.
8. Add sudoers entry for `sayo-led`.
9. Install the autostart desktop file.
10. Configure `logrotate`.
11. Run `black` on changed project files after edits.
12. Reboot and watch `stereo-display.log`.

Current Backlog / Future Enhancements

Possible future improvements:

- Cache Spotify metadata corrections by normalized artist/title to reduce repeated Spotify searches.
- Add a manual override database for records ACRCLOUD cannot identify correctly.
- Improve album-selection heuristics further if Spotify returns multiple plausible releases.
- Add an HDMI-specific UI if moving away from the CRT.
- Investigate Pi 5 migration for more headroom.
- Revisit waveform/spectrum visualization only if the Sony VT-M5 repair path fails.
- More formal service management instead of desktop autostart.